TP 4 : Recherche d'Information Booléenne Étendue et Floue

5 mars 2026

Objectif : Implémenter trois modèles de recherche avancés :
- Modèle booléen étendu (p-norm)
- Modèle flou - Méthode 1 (Lukasiewicz)
- Modèle flou - Méthode 2 (Kraft et al.)

avec interface graphique permettant de choisir le modèle de recherche.

1 Préparation du projet

1.1 Création du projet

Avec PyCharm, créez un nouveau projet nommé `TP4_MODELE_ETENDU`.

1.2 Récupération du TP2

1. Téléchargez le code du TP2 depuis le lien git :

   `https://github.com/Anwer-kal/TP2-Indexation`

2. Décompressez l'archive et copiez le dossier `TP2-Indexation-main` dans votre projet

1.3 Installation des bibliothèques

Méthode 1 : Interface PyCharm

```
File > Settings > Project > Python Interpreter > [+] > Rechercher :
- PyPDF2
- nltk
- pandas
```

Note : Tkinter est déjà installé par défaut avec Python et PyCharm.

1.4 Création de la structure

Créez l'arborescence suivante :

```
TP4_MODELE_ETENDU/

 modeles/
    __init__.py
    modele_etendu.py
    modele_lukasiewicz.py
    modele_kraft.py

 index_util.py    # Charge les docs du TP2
 recherche.py     # Moteur avec 3 modèles
 interface.py     # Interface graphique
 main.py          # Point d'entrée
 TP2-Indexation-main/
```

2 Rappel des formules mathématiques

Modèle booléen étendu (p-norm avec p=2)

$\mathrm{R}(\mathrm{d}_j, q_{or}) = \left(\frac{w_1^p + w_2^p + \ldots + w_{mj}^p}{m}\right)^{\frac{1}{p}}$
$R(d_j, q_{and}) = 1 - \left(\frac{(1-w_1)^p + (1-w_2)^p + \ldots + (1-w_{mj})^p}{m}\right)^{\frac{1}{p}}$
$R(d_j, q_{not}) = 1 - R(d_j, q)$ avec $p = 2$, $m =$ nombre de termes dans la requête

Méthode 1 : Lukasiewicz (flou)

$\mathrm{R}(\mathrm{d},\ \mathrm{t}_i) = a_i$
$R(d, q_1 \wedge q_2) = R(d, q_1) \times R(d, q_2)$
$R(d, q_1 \vee q_2) = R(d, q_1) + R(d, q_2) - R(d, q_1) \times R(d, q_2)$
$R(d, \neg q_1) = 1 - R(d, q_1)$

Méthode 2 : Kraft et al. (min-max)

$\mathrm{R}(\mathrm{d},\ \mathrm{q}_1 \wedge q_2) = \min(R(d, q_1), R(d, q_2))$
$R(d, q_1 \vee q_2) = \max(R(d, q_1), R(d, q_2))$
$R(d, \neg q_1) = 1 - R(d, q_1)$

3 Fichier : `modeles/__init__.py`

But : Rendre les modèles importables depuis le dossier modeles

```python
from .modele_etendu import ModeleEtendu
from .modele_lukasiewicz import ModeleLukasiewicz
from .modele_kraft import ModeleKraft
```

4 Fichier : `modeles/modele_etendu.py`

But : Implémente le modèle booléen étendu avec p=2

```python
import math

class ModeleEtendu:
    def __init__(self, p=2):
        self.p = p

    def ou(self, poids):
        if not poids:
            return 0
        return (sum(w**self.p for w in poids) / len(poids)) ** (1/self.p)

    def et(self, poids):
        if not poids:
            return 0
        return 1 - (sum((1-w)**self.p for w in poids) / len(poids)) ** (1/self.p
    )

    def non(self, w):
        return 1 - w
```

5 Fichier : modeles/modele_lukasiewicz.py

But : Implémente le modèle flou de Lukasiewicz

```python
class ModeleLukasiewicz:
    def et_liste(self, poids):
        if not poids: return 0
        r = 1
        for w in poids: r *= w
        return r
    def ou_liste(self, poids):
        if not poids: return 0
        r = 0
        for w in poids: r = r + w - r*w
        return r
    def non(self, w): return 1 - w
```

6 Fichier : modeles/modele_kraft.py

But : Implémente le modèle flou de Kraft (min-max)

```python
class ModeleKraft:
    def et_liste(self, poids):
        return min(poids) if poids else 0

    def ou_liste(self, poids):
        return max(poids) if poids else 0

    def non(self, w):
        return 1 - w
```

7 Fichier : index_util.py

But : Charge les documents PDF et calcule les poids TF-IDF

```python
import os
import sys
import re

sys.path.append("TP2-Indexation-main")
from etape1_pdf import extract_words
from etape2_stopwords import remove_stopwords
from etape4_tf import compute_tf
from etape5_idf import compute_idf
from etape6_tfidf import compute_tfidf


class IndexUtil:
    def __init__(self):
        self.docs = {}  # nom -> mots
        self.tfidf = {}  # nom -> {terme: tfidf}
        self.tf = {}  # nom -> {terme: tf}
        self.idf = {}  # terme -> idf

    def construire(self):
        dossier = "TP2-Indexation-main/pdfs-main"

        # 1. Lire tous les documents
        docs_mots = {}
        for f in os.listdir(dossier):
```

```python
            if f.endswith(".pdf"):
                try:
                    mots = extract_words(f"{dossier}/{f}")
                    mots = remove_stopwords(mots)
                    mots = [mot for mot in mots if not mot.isdigit()]
                    if mots:
                        self.docs[f] = mots
                        docs_mots[f] = mots
                        self.tf[f] = compute_tf(mots)
                except:
                    pass

        # 2. Calculer IDF sur tous les docs
        self.idf = compute_idf(docs_mots)

        # 3. Calculer TF-IDF pour chaque doc
        for doc in self.docs:
            self.tfidf[doc] = compute_tfidf(self.tf[doc], self.idf)

    def get_poids(self, terme, doc):
        """Retourne le poids TF-IDF du terme dans le doc"""
        return self.tfidf.get(doc, {}).get(terme, 0)
```

8 Fichier : recherche.py

But : Moteur de recherche avec les 3 modèles

```python
from modeles import *

class Recherche:
    def __init__(self, index):
        self.index = index
        self.modeles = {
            "etendu": ModeleEtendu(),
            "lukasiewicz": ModeleLukasiewicz(),
            "kraft": ModeleKraft()
        }
        self.modele_courant = "etendu"

    def set_modele(self, nom):
        if nom in self.modeles:
            self.modele_courant = nom
            return True
        return False

    def rechercher(self, requete):
        mots = requete.lower().split()
        if not mots:
            return []

        modele = self.modeles[self.modele_courant]
        resultats = []

        for doc in self.index.docs:
            # Poids TF-IDF des mots de la requ te dans ce document
            poids = [self.index.get_poids(m, doc) for m in mots]

            if self.modele_courant == "etendu":
                s_ou = modele.ou(poids)
                s_et = modele.et(poids)
                score = (s_ou + s_et) / 2
            elif self.modele_courant == "lukasiewicz":
```

```
                s_et = modele.et_liste(poids)
                s_ou = modele.ou_liste(poids)
                score = (s_et + s_ou) / 2
            else:  # kraft
                s_et = modele.et_liste(poids)
                s_ou = modele.ou_liste(poids)
                score = (s_et + s_ou) / 2

            if score >= 0:
                resultats.append((doc, score))

        resultats.sort(key=lambda x: -x[1])
        return resultats
```

9 Fichier : interface.py

But : Interface graphique simple avec Tkinter

```
import tkinter as tk
from tkinter import ttk, scrolledtext


class Interface:
    def __init__(self, recherche):
        self.r = recherche
        self.f = tk.Tk()
        self.f.title("TP4 - 3 Mod les")
        self.f.geometry("800x600")

        tk.Label(self.f, text="TP4 : Recherche avec 3 mod les",
                 font=("Arial", 14)).pack(pady=5)

        frame_modele = tk.Frame(self.f)
        frame_modele.pack(pady=5)
        tk.Label(frame_modele, text="Mod le:").pack(side=tk.LEFT)
        self.cb = ttk.Combobox(frame_modele,
                               values=["etendu", "lukasiewicz", "kraft"],
                               state="readonly")
        self.cb.pack(side=tk.LEFT, padx=5)
        self.cb.set("etendu")
        self.cb.bind('<<ComboboxSelected>>', self.changer_modele)

        tk.Label(self.f, text="Requ te:").pack()
        self.entry = tk.Entry(self.f, width=60, font=("Arial", 12))
        self.entry.pack(pady=5)
        self.entry.bind("<Return>", lambda e: self.chercher())

        tk.Button(self.f, text="Chercher", command=self.chercher,
                  bg="green", fg="white").pack(pady=5)

        self.txt = scrolledtext.ScrolledText(self.f, height=20, font=("Courier", 10))
        self.txt.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        self.f.mainloop()

    def changer_modele(self, event):
        self.r.set_modele(self.cb.get())
        self.txt.insert(tk.END, f"Mod le chang : {self.cb.get()}\n")

    def chercher(self):
        req = self.entry.get().strip()
```

```
        if not req:
            return

        self.txt.delete(1.0, tk.END)
        self.txt.insert(tk.END, f"Requ te: {req}\n")
        self.txt.insert(tk.END, f"Mod le: {self.cb.get()}\n")
        self.txt.insert(tk.END, "=" * 50 + "\n\n")

        res = self.r.rechercher(req)

        if res:
            for i, (doc, score) in enumerate(res, 1):
                self.txt.insert(tk.END, f"{i:2d}. {doc} - score: {score:.4f}\n")
        else:
            self.txt.insert(tk.END, "Aucun r sultat\n")
```

10 Fichier : `main.py`

But : Point d'entrée du programme

```
from index_util import IndexUtil
from recherche import Recherche
from interface import Interface

def main():
    print("TP4 - D marrage...")
    index = IndexUtil()
    index.construire()
    recherche = Recherche(index)
    Interface(recherche)

if __name__ == "__main__":
    main()
```

Comment exécuter

```
# 1. Mettre le dossier TP2-Indexation-main dans le projet
# 2. Lancer le programme
python main.py
```

Question d'amélioration

À faire : Proposez et implémentez une amélioration de l'interface graphique.
Idées possibles :

- Ajouter des boutons pour les opérateurs (ET, OU, NON, parenthèses)
- Afficher un aperçu du contenu des documents trouvés

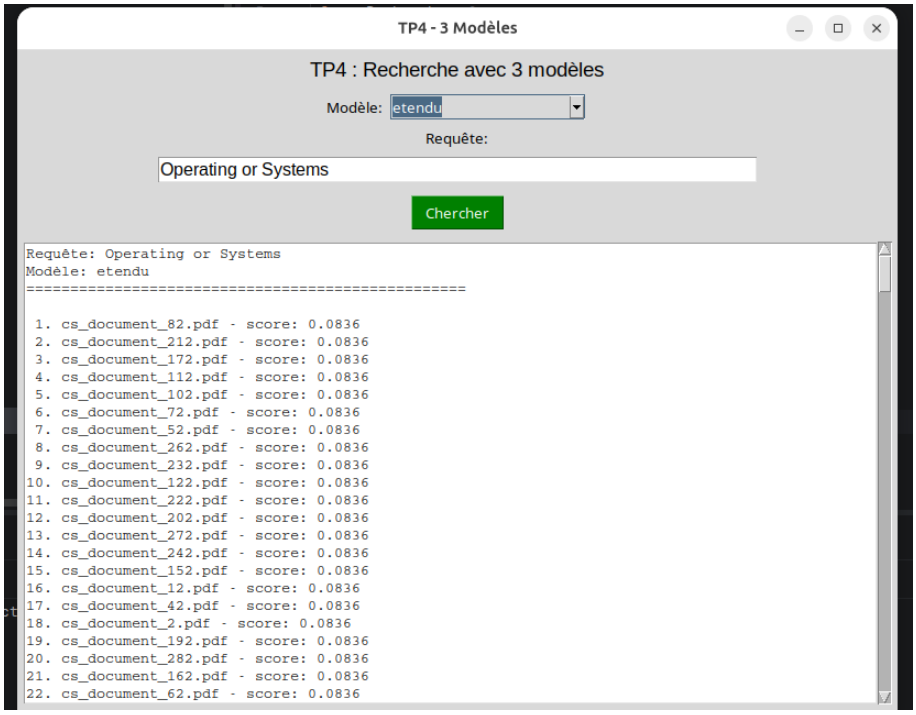

FIGURE 1 – Exemple

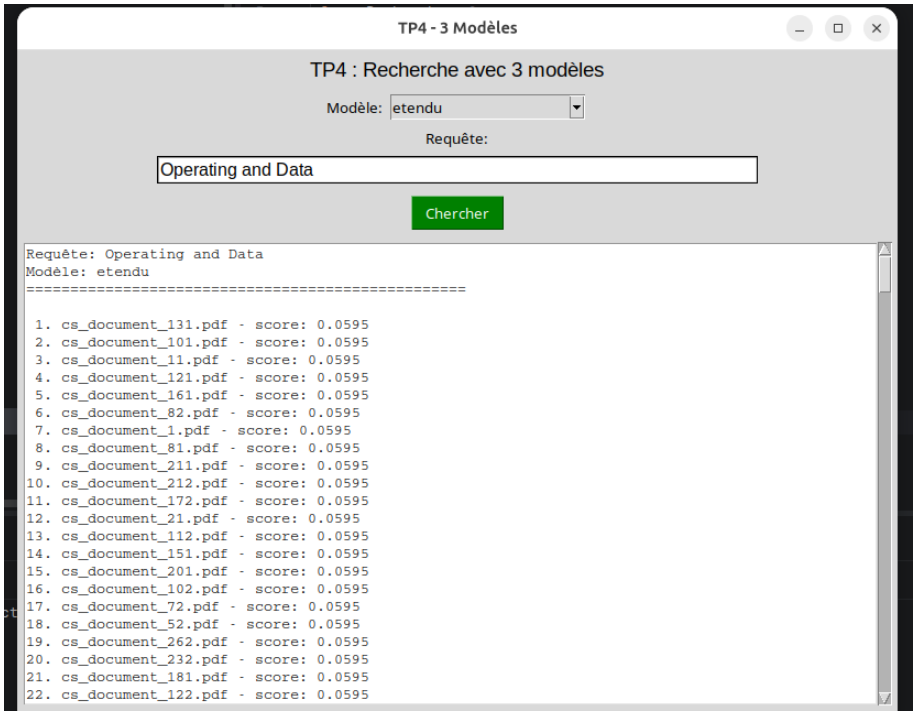

FIGURE 2 – Exemple